Module 11 — Integration Task: Automated Evaluation Harness for the NLP + KG + RAG Stack

Due: Thursday end of day. Resubmissions are accepted through Saturday of the assignment week.

The Lab made the stack *observable*. The Integration makes the stack *measurable*. You will author three evaluation harnesses — one per endpoint — that run against the live M11-instrumented stack, score answers against labeled fixtures, and produce a single Markdown report with three headline numbers (NER (named-entity recognition) F1, NL (natural-language) → Cypher exact-match (EM) accuracy, RAG (retrieval-augmented generation) grounding rate) plus three derived signals from `/metrics` (per-endpoint p95 latency, error rate, total request count). The Integration's three target endpoints span the NLP (natural-language processing), KG (knowledge graph), and RAG components of the M10 stack.

This is the Module 12 capstone deliverable in miniature. The capstone monitoring + evaluation requirement *is* this harness: the capstone team will divide work assuming every member has built one. By Thursday night, you have.

This Integration is graded **individually**. (Module 10's Integration was the team week; M11 returns to individual cadence so every learner produces individual evidence of harness authorship.)

Setup

Accept the Integration template repo at `https://github.com/LevelUp-Applied-AI/m11-ill` — click **"Use this template"** → **"Create a new repository"**. Clone:

```
git clone https://github.com/<your-username>/m11-ill.git
cd m11-ill
git checkout -b integration-ll-compose-evaluation
```

Use **Python 3.11** for this template — `pydantic==2.6.0` does not build on 3.13. Create and activate a 3.11 venv before installing dependencies:

```
python3.11 -m venv .venv
source .venv/bin/activate
pip install -r requirements.txt
```

If you have not yet installed `prometheus-client`, go back to the Setup: install before the concept session section of the reading.

The Integration template ships with the M11-instrumented backend pre-baked into `api/` — you are **not** required to have landed the Lab to start the Integration. Bring up the stack:

```
cp compose/.env.example compose/.env
docker compose -f compose/docker-compose.yml up -d
# Full /ready: ~3min1 takes ~30 s to come up on a cold start.
for i in $(seq 1 60); do
  if curl -fs8 http://localhost:8000/ready> >/dev/null 2>&1; then
    echo "Stack ready"; break
  fi
  sleep 1
done
bash compose/seed_neo4j.sh
bash compose/seed_weaviate.sh
```

What ships in the starter

```
m11-ill/
├── api/                                pre-built M11-instrumented backend (so Integra
├── compose/
│   ├── docker-compose.yml              M10 stack with M11 api
│   ├── .env.example
│   ├── seed_neo4j.sh
│   └── seed_weaviate.sh
├── data/
│   ├── nmr_conll30.json                30 CoNLL-2003 (Conference on Natural Language
│   ├── kg_questions.json               25 NL→Cypher pairs (NYB-bounded schema)
│   └── rag_questions.json              20 RAG questions with gold-citation chunk-ids
├── eval_ner.py                         TODO: NER F1 harness
├── eval_kg.py                          TODO: NL→Cypher exact-match harness
├── eval_rag.py                         TODO: RAG grounding-rate harness
├── eval_runner.py                      TODO: orchestrator + report assembly
├── lib/
│   ├── __init__.py
│   ├── metrics_reader.py               PROVIDED helper for /metrics parsing
│   ├── ner_scorer.py                   TODO: F1 scoring functions
│   ├── cypher_normalizer.py            TODO: whitespace + keyword normalization
│   └── grounding_scorer.py             TODO: decline detection + cited-id verificatio
├── tests/
│   ├── test_evaluation_harness.py      autograder
│   ├── test_learner_methodology.py     learner-written-test placeholder
│   └── conftest.py
├── reports/                            evaluation-report.md lands here
└── requirements.txt
```

The `lib/metrics_reader.py` helper is **provided**, not TODO. It wraps `httpx.get("/metrics")` and `prometheus_client.parser.text_string_to_metric_families` and exposes `get_request_count(path)`, `get_error_rate(path)`, `get_p95_latency(path)`. Read its code so you understand the parser output, but you do not need to modify it.

Tasks

Step 1 — Bring up the stack

```
docker compose -f compose/docker-compose.yml up -d
# Full /ready: ~3min1 takes ~30 s to come up on a cold start.
for i in $(seq 1 60); do
  if curl -fs8 http://localhost:8000/ready> >/dev/null 2>&1; then
    echo "Stack ready"; break
  fi
  sleep 1
done
bash compose/seed_neo4j.sh
bash compose/seed_weaviate.sh
```

Step 2 — Implement `eval_ner.py`

Load `data/nmr_conll30.json` (30 documents, each with `document_id`, `text`, `gold_entities`.) For each document, `POST /extract` with `{"text": <doc text>}` and read the returned entities. Score using `lib/ner_scorer.py`:

- An entity is a **true positive** if and only if a gold entity with the *same* `document_id`, `entity_text`, and `entity_label` exists in the gold set.
- A predicted entity with no matching gold entity is a **false positive**.
- A gold entity with no matching predicted entity is a **false negative**.
- **Precision** = TP / (TP + FP); **Recall** = TP / (TP + FN); **F1** = 2 P R / (P + R).
- **Micro-average** across all 30 documents — sum TPs/FPs/FNs across all documents, then compute once.

The methodology paragraph shipped in the `eval_ner.py` module docstring is the canonical operational text for this Integration's NER scoring — it states the same methodology as reading §12, in its own canonical form. Leave the shipped docstring as it is; do **not** overwrite it with the reading-§12 prose. The `lib/ner_scorer.py` functions are testable in isolation: `score_document(pred, gold)` returns `(tp, fp, fn)`, and `compute_micro_f1(predictions_by_doc, gold_by_doc)` returns `(precision, recall, f1)` after summing TP/FP/FN across the gold document set.

Write per-document results and the headline F1 to `reports/evaluation-report.md` (your `eval_runner.py` will orchestrate the writes — for now, return the data structures).

Step 3 — Implement `eval_kg.py`

Load `data/kg_questions.json` (25 NL→Cypher pairs, each with `question`, `gold_cypher`.) For each question, `POST /kg/query` with `{"question": q}`. Three response cases:

- **200 with a `cypher` field.** Normalize both the returned `cypher` and the `gold_cypher` via `lib/cypher_normalizer.py` (collapse whitespace; uppercase Cypher keywords from the list `MATCH, RETURN, WHERE, OPTIONAL, WITH, LIMIT, ORDER BY`.) Score 1 if normalized-equal, 0 otherwise.
- **422 `UnsupportedQueryError`.** The mapper recognized the question as out-of-intent. **Exclude from the denominator** — the metric is exact-match on *supported* questions.
- **5xx.** Score 0 (the harness counts it as an answered failure) and surface in the report's sample-failures list.

Exact-match accuracy = (sum of 1s) / (count of supported questions). The shipped `eval_kg.py` docstring is the canonical operational text — it states the same methodology as reading §13 in its own form. Leave the shipped docstring as it is.

Step 4 — Implement `eval_rag.py`

Load `data/rag_questions.json` (20 RAG questions, each with `question_id`, `question`, `k`, `gold_citation_chunk_ids` — a flat array of chunk-id strings. The gold field is informational; the grounding metric is computed against the retrieval candidate set, not against gold.) For each question, `POST /rag/answer` with `{"question": q, "k": k}`. Then:

- If the response `answer` is the canonical decline string `"I cannot answer this from the available sources."`, **exclude from the denominator**.
- Otherwise, read the candidate set from the `/rag/answer` response's `retrieved` field (each entry is a `{"chunk_id": ...}` object).
- Score as **grounded** iff (a) the response's `citations` array has length ≥ 1 AND (b) every cited `chunk_id` is in the candidate set.

Grounding rate = grounded / answered. The shipped `eval_rag.py` docstring is the canonical operational text — it states the same methodology as reading §14 in its own form. Leave the shipped docstring as it is.

Step 5 — Implement `eval_runner.py`

The orchestrator. Call `eval_ner.run()`, `eval_kg.run()`, `eval_rag.run()`. Then use `lib/metrics_reader.py` to compute, per endpoint, the p95 latency, the error rate, and the total request count from `/metrics`. Assemble `reports/evaluation-report.md` with:

- **Headline section.** Three numbers: NER F1, NL→Cypher EM, RAG grounding rate.
- **Per-endpoint methodology paragraphs.** Copied verbatim from the three harness docstrings.
- **Per-endpoint per-question tables.** One row per fixture item with the gold, the prediction, and the per-item pass/fail.
- **Sample failures.** At least 3 per endpoint, with gold versus predicted side-by-side.
- **Derived `/metrics` signals.** Per-endpoint p95, error rate, total request count.

The report is a single Markdown file. Commit it.

Step 6 — Write your tests in `tests/test_learner_methodology.py`

The starter has `pytest.fail("Not implemented")` placeholders. Write at least **three** test functions, each with at least one `assert`. Required coverage:

1. The NER scorer produces F1 = 1.0 on a synthetic 3-document fixture where predictions exactly match gold; F1 = 0.0 when predictions are disjoint from gold.
2. The Cypher normalizer collapses runs of whitespace to a single space *and* uppercases the keyword list — verify both transformations on a paired sample.
3. The grounding-rate scorer correctly excludes the canonical decline string from the denominator — given 10 answered + 5 declined items, denominator = 10.

Run:

```
pytest tests/test_learner_methodology.py -v
```

Step 7 — Run the pipeline

```
python eval_runner.py --output reports/evaluation-report.md
```

The autograder will assert the report exists, parses, contains three headline numbers, and that each headline meets its floor (NER F1 ≥ 0.65, NL→Cypher EM ≥ 0.80, RAG grounding rate ≥ 0.85).

Step 8 — Tear down

```
docker compose -f compose/docker-compose.yml down -v
```

Submission

Commit `reports/evaluation-report.md`, your scorer implementations, and your tests. Push, open a PR within your fork. Your PR description must include:

1. Confirmation that `python eval_runner.py` produces `reports/evaluation-report.md` and all three floors are met.
2. The three headline numbers as Markdown copied from your report.
3. A short paragraph (~150 words) reflecting on one methodology decision you made — for example: why you chose micro-average for NER F1 instead of macro; how you handled the Cypher whitespace normalization edge case where the gold had a trailing semicolon; how your grounding scorer treated a response that cited the same chunk_id twice.
4. Paste your PR URL into TalentLMS → Module 11 → Integration 11 to submit this assignment.

Common pitfalls

- **Methodology drift.** The docstring says micro-average; the implementation accidentally macro-averages. The headline number changes; the autograder may still pass on a high-baseline fixture, but the reviewer can see the discrepancy. Treat the methodology paragraph as the contract; the code implements it.
- **Including the decline path in the grounding denominator.** Your headline number drops; learners hit this every cohort. The reading §14 calls this out — re-read.
- **422 included in NL→Cypher denominator.** The mapper's `UnsupportedQueryError` is *not* a wrong answer; it is an out-of-intent recognition. Exclude.
- **`https` timeout too short for `/rag/answer`.** Default 5 s; the endpoint can spike on cold cache. Pass `timeout=60.0`.
- **Re-running the retrieval call with the wrong `k`.** The grounding check requires the same `k` as the answer call. If you used `k=5` to generate and `k=10` to verify, you over-credit the answer.
- **Report missing the methodology paragraphs.** The methodology for each metric is the docstring of its `eval_*.py` harness — that is the canonical text. Your `eval_runner.py` should copy each docstring into the report rather than retyping the paragraph from memory.
- **Empty per-question table.** The harness silently caught an exception on the first fixture item. Add a `--verbose` flag that prints every request/response pair so you see the first failure.

Challenge Extensions

Optional. Each tier extends the base Integration with a new concept; no starter files for tiers.

Tier 1 — Bootstrap confidence intervals on each headline

The base Integration reports three point estimates. Tier 1 extends each harness with a **95% bootstrap confidence interval (CI)** by (i) resampling the per-question outcomes with replacement 1000 times; (ii) recomputing the metric on each resample; (iii) reporting the 2.5th and 97.5th percentiles. Add a `## Confidence Intervals` section to the report showing the three intervals. Write one test verifying the bootstrap function on a synthetic fixture — 10 outcomes all correct should give a confidence interval of [1.0, 1.0]; 5 correct of 10 should give a confidence interval that brackets 0.5. (Note: CI here means *confidence interval*, not *continuous integration* — context matters in this paragraph.) Substantive concept: statistical uncertainty in evaluation. A quality number alone is misleading without a sense of its variance — particularly on small fixtures (20–30 items) like the ones this Integration uses.

Tier 2 — Author a custom evaluation fixture for one endpoint

The base Integration uses three vendored fixtures. Tier 2 asks you to author a **new 20-question fixture** for one endpoint (NER, KG, or RAG) drawn from a domain *not* represented in the vendored fixtures (e.g., legal-entity recognition; sports-stats Cypher against a Weaviate index you re-seed with a documented sports-history corpus). The fixture must include (a) `fixture-spec.md` documenting source, labeling methodology, ambiguity-handling rules, and a paragraph on why your fixture exercises a different stress on the endpoint than the vendored one; (b) the fixture file in the same schema as the vendored one; (c) a re-run of the harness against your new fixture with the report updated to show both numbers side-by-side. Substantive concept: *fixture authorship* is its own discipline — design choices in the fixture determine what the harness can and cannot tell you about the service.

Tier 3 — Build a failure-driven regression-test generator

Implement an abstraction `RegressionCorpus` that (a) reads the failures from your base evaluation report; (b) for each failure, generates a pytest test function (via a code generator, not a string template) that asserts the *current behavior* the harness measured — so that *if a future code change improves that case, the test breaks, signaling a regression catch*; (c) outputs `tests/test_regression_corpus.py` with one test per failure; (d) is config-driven via `regression-config.yaml` (which endpoints to include, severity threshold, output naming); (e) ships with `tier-3-design.md` discussing when regression-from-failure tests are an asset (preventing silent quality degradation) and when they are a trap (locking in current-implementation bugs as expected behavior). Substantive concept: automated test-corpus generation as a quality discipline — a real engineering pattern from production ML ops. Requires designing an API (application programming interface, here meaning the generator's function-level contract), building a reusable abstraction (the config schema), and reasoning about second-order effects.